

High Performance Computing – OpenMP Lab Assignment

Orazgali Azamat, BDA-2402

All programs were compiled with `g++ -O2 -std=c++11 -fopenmp` (MinGW-w64 16.1.0). The machine has 24 logical processors. Exception: the race condition programs were compiled with `-O0` (explained in Part 5).

Part 1: Your First OpenMP Program

Source code: `openmp_test.cpp`. Output with 4 threads (one run):

Hello from thread Hello from thread Hello from thread 1 of 4

Hello from thread 3 of 4

0 of 4

2 of 4

Output with 1 thread:

Hello from thread 0 of 1.

With 2 threads:

Hello from thread 0 of 2Hello from thread 1 of 2

1. How many times is the code inside the parallel region executed when four threads are used?

Four times, once by each thread.

2. What values are returned by `omp_get_thread_num()`?

0, 1, 2 and 3 (each thread gets a different number, starting from 0).

3. Does the order in which the threads print their messages remain the same between different executions?

No. The order changes from run to run, and the lines can even get mixed because the threads write to `std::cout` at the same time.

4. Is the code before and after the parallel region executed by every thread or only by the original thread?

Only by the original (master) thread.

Part 2: Parallel Vector Computation

Source code: vector_compute.cpp. The loop for $C[i] = 2*A[i] + B[i]$ was parallelized with `#pragma omp parallel for`. Output:

$C[0] = 0$

$C[N-1] = 1.25e+06$

1. Can different iterations of this loop be executed independently?

Yes. Each iteration only writes its own $C[i]$ and only reads $A[i]$ and $B[i]$.

2. Which OpenMP directive did you use?

`#pragma omp parallel for`

3. Is the loop variable i shared between threads or private to each thread?

Private. OpenMP makes the loop variable of a parallel for private to each thread.

4. Are arrays A , B , and C shared by the threads?

Yes, they are shared. All threads work on the same arrays, but on different elements.

5. Why is this loop suitable for parallel execution?

There are no dependencies between iterations and each iteration does the same amount of work, so the iterations can be divided among threads without any communication.

Part 3: Observing Work Distribution

Source code: work_distribution.cpp ($N = 20$, 4 threads). The printed lines from the threads get mixed together, so the iterations of each thread were collected with a small helper program that stores `omp_get_thread_num()` for every iteration. The result was the same in 3 runs:

Thread	Iterations
0	0, 1, 2, 3, 4
1	5, 6, 7, 8, 9
2	10, 11, 12, 13, 14
3	15, 16, 17, 18, 19

1. Is every iteration executed exactly once?

Yes. All 20 iterations (0 to 19) appear once.

2. Does every thread receive approximately the same amount of work?

Yes. Each thread gets 5 iterations.

3. Are consecutive iterations generally assigned to the same thread when the default scheduling strategy is used?

Yes. With the default (static) scheduling the iterations are divided into consecutive blocks, one block per thread.

Part 4: Parallel Reduction

Source code: reduction.cpp. The completed line is:

```
#pragma omp parallel for reduction(+:sum)
```

With $A[i] = 1$ and $N = 1,000,000$ the output was: $\text{Sum} = 1e+06$ (which is 1,000,000, as expected).

1. Why is updating one shared sum variable directly unsafe?

Several threads may read and write sum at the same time. Then one thread can overwrite the update of another thread and some additions are lost (a race condition).

2. What does the OpenMP reduction operation do conceptually?

Each thread gets its own private copy of the variable, accumulates its own partial result, and at the end all the partial results are combined into the final value using the given operator (here +).

3. Does every thread initially work with the same shared accumulator or with its own partial result?

With its own partial result (a private copy initialized to 0 for +).

4. When are the partial results combined?

At the end of the parallel loop, after the threads finish their iterations.

Part 5: Race Condition

Source codes: race_condition.cpp (original) and race_atomic.cpp (with #pragma omp atomic). 4 threads, $N = 1,000,000$. When compiled with -O2 the optimizer almost hides the problem: 14 of 15 runs still gave 1000000 and one gave 750000. For this reason the experiment below was compiled with -O0, where the race appears every time.

Run	Value of counter (no synchronization)	Value of counter (with atomic)
1	261763	1000000
2	258936	1000000
3	254340	1000000
4	252730	1000000
5	376972	1000000

1. Why can the original program produce an incorrect value?

The threads update the shared variable counter at the same time without coordination, so some increments are lost and the final value is smaller than 1,000,000. It is also different from run to run.

2. Why is the statement `counter++`; not necessarily one indivisible operation?

It consists of three steps: read the value from memory, add 1, write the result back. Another thread can read the same old value between these steps, so both write the same new value and one increment disappears.

3. What changes after adding `#pragma omp atomic`?

The read-modify-write of counter is done as one indivisible operation, so only one thread updates it at a time. The result is always 1,000,000.

4. Does synchronization have a performance cost?

Yes. Threads must wait for each other and atomic operations are slower than normal ones, so part of the parallel speedup is lost.

Part 6: Static and Dynamic Scheduling

Source code: scheduling.cpp (N = 100000, 4 threads, time measured with `omp_get_wtime()`). Three runs were made; the table shows the average.

Scheduling Method	Execution Time (seconds)
Static	0.0217
Dynamic	0.0217

1. What does `schedule(static)` do?

It divides the iterations into equal blocks before the loop starts and assigns one block to each thread.

2. What does `schedule(dynamic,10)` do?

The iterations are divided into chunks of 10. Each thread takes a new chunk when it finishes its previous one, so the assignment is decided during execution.

3. Why might dynamic scheduling be useful when different iterations require different amounts of work?

Threads that get cheaper iterations finish earlier and can take more chunks, so the work stays balanced and no thread is idle while another is still busy.

4. Does dynamic scheduling introduce additional scheduling overhead?

Yes, because threads must request chunks at run time. In my test the two times were

equal, because the work per iteration repeats every 1000 iterations and every static block of 25000 iterations contains the same mix of work, so static was already balanced.

Part 7: Performance Experiment

Source code: performance.cpp. N was increased to 20,000,000 so that the time can be measured. Only the parallel loop is timed. Three runs for each thread count were made; the table shows the average.

Threads	Execution Time (seconds)	Speedup
1	0.0183	1.00
2	0.0143	1.28
4	0.0103	1.77

1. Does execution time decrease when more threads are used?

Yes, it decreased from 0.0183 s with 1 thread to 0.0103 s with 4 threads.

2. Is the speedup exactly equal to the number of threads?

No. With 2 threads it was 1.28 and with 4 threads 1.77.

3. Give one reason why a program may not obtain ideal speedup.

This loop does very little arithmetic per element and mostly reads and writes memory, so the memory bandwidth limits it. Other reasons are thread creation and synchronization overhead and the serial part of the program.

4. Would you expect increasing the number of threads indefinitely to continue improving performance?

No. After some point the overhead, the memory bandwidth limit, the serial part of the program (Amdahl's law) and the limited number of physical cores stop the improvement, and too many threads can even make the program slower.

Part 8: Short Discussion

1. What is the difference between a serial region and a parallel region in OpenMP?

A serial region is executed by only one thread (the master thread). A parallel region is executed by a team of threads at the same time.

2. What is the purpose of `#pragma omp parallel for`?

It creates a team of threads and divides the iterations of the following for loop among them.

3. What is the difference between a shared variable and a private variable?

A shared variable is one memory location seen by all threads. A private variable has a separate copy for every thread.

4. Why is a reduction useful for operations such as summation?

Each thread sums its own part without interfering with others, and the partial sums are combined safely at the end. It is correct and does not need slow synchronization on every addition.

5. What is a race condition?

A situation where several threads access the same variable at the same time, at least one of them writes, and the result depends on the unpredictable order of execution.

6. Why can synchronization reduce parallel performance?

Threads have to wait for each other, so work that could run in parallel is executed one thread at a time, and the synchronization operations themselves cost time.

7. What is the difference between static and dynamic scheduling?

Static scheduling assigns the iterations to threads in advance, in fixed blocks, with little overhead. Dynamic scheduling gives chunks of iterations to threads while the loop runs, which balances uneven work better but has more overhead.